

基于 NSGA-II 多目标优化的 CNN-BiLSTM 风速预测模型

王晨淇

2025 年 11 月 14 日

1 问题描述

风速作为典型的非线性、非平稳时序数据，其变化受地形、气候等多因素耦合影响，表现出强波动性与复杂的时空关联性。精准的风速预测是风力发电调度、电网稳定运行及能源规划的关键前提。现有风速预测模型存在以下局限性：(1) 单一目标优化策略仅关注预测精度的提升，易导致模型结构冗余及参数规模过大；(2) 单一网络结构难以同时兼顾局部特征提取与长程时序依赖捕捉能力。本文提出一种基于改进 NSGA-II 算法的多目标优化 CNN-BiLSTM 风速预测模型。优化目标定义为：

- 1) **最小化预测误差**： $\min f_1 = \text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$ ；
- 2) **最小化模型复杂度**： $\min f_2 = \text{Complexity} = \sum_{l=1}^L \text{Param}_l$ ，其中 Param_l 为第 l 层可训练参数数量。

通过 NSGA-II 算法搜索帕累托最优超参数组合，有效解决传统模型在预测精度与复杂度之间的权衡问题。

2 基于 NSGA-II 多目标优化的 CNN-BiLSTM 风速预测模型

2.1 方法框架与问题建模

2.1.1 风速预测模型自动优化问题描述

NSGA-II-CNN-BiLSTM 的构建框架如下所示，与 BiLSTM、GRU、CNN 和 Transformer 等不同模型架构进行对比，这些模型均采用 NSGA-II 进行多目标优化。

针对风速预测场景（即风速数据集 $D = \{D_{\text{train}}, D_{\text{valid}}, D_{\text{test}}\}$ ），模型自动化设计问题可描述为如下多目标优化问题，即在满足模型复杂度约束条件下同时最小化预测误差

与模型参数量:

$$\begin{aligned} \min_{\mathbf{x} \in \mathcal{S}} \quad & \begin{cases} f_1(\mathbf{x}) = \text{RMSE}(\mathbf{x}) = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2} \\ f_2(\mathbf{x}) = \text{Complexity}(\mathbf{x}) = \sum_{l=1}^L \text{Param}_l(\mathbf{x}) \end{cases} \\ \text{s.t.} \quad & \text{Complexity}(\mathbf{x}) \leq C_{\max} \end{aligned} \quad (1)$$

其中 $\mathbf{x}$ 表示待优化模型架构及其超参数; $\mathcal{S}$ 表示超参数搜索空间; $\text{RMSE}(\mathbf{x})$ 表示模型 $\mathbf{x}$ 在验证集上的均方根误差; $\text{Complexity}(\mathbf{x})$ 表示模型可训练参数总量; $C_{\max}$ 表示预设的模型复杂度上限。在架构搜索阶段, 使用训练集 D_{train} 训练 $\mathbf{x}$ 后, 于验证集 D_{valid} 上计算 $\text{RMSE}(\mathbf{x})$ 和 $\text{Complexity}(\mathbf{x})$; 在测试阶段, 使用 D_{train} 训练最优模型后, 于测试集 D_{test} 上统计 MAE、MAPE、R 值等性能指标。

2.1.2 NSGA-II-CNN-BiLSTM 方法总体框架

NSGA-II-CNN-BiLSTM 的基本框架如下, 总体流程包含三个主要部分: 种群初始化、适应度评估和基于 NSGA-II 的多目标优化流程, 输入参数为迭代次数 G , 种群规模 N , 交叉率 θ 和变异率 σ 。系统采用**多进程并行**架构, 每个模型类型在独立进程中运行, 充分利用多 GPU 资源加速优化。整体流程如下:

- 1) **初始化**: 为每个模型类型随机生成满足约束的 N 个个体种群
- 2) **评估**: 在独立进程中并行评估种群, 每个个体解码后训练模型 **20** 轮, 计算 RMSE 和复杂度
- 3) **非支配排序**: 构建 Pareto 前沿层级, 区分不同优化程度的解
- 4) **拥挤度计算**: 计算标准拥挤度距离, 保持种群多样性
- 5) **锦标赛选择**: 基于非支配排序顺序和拥挤度进行选择操作
- 6) **差异化交叉变异**: 针对超参数类型采用不同的进化算子
- 7) **环境选择**: 合并父代子代, 基于非支配排序和拥挤度选择最优 N 个进入下一代
- 8) **迭代**: 重复 2-7 步直至最大代数 G , 最终输出 Pareto 最优解集

2.2 混合编码方案与种群初始化

NSGA-II-CNN-BiLSTM 采用**动态混合编码策略**, 核心逻辑与代码完全一致: 种群存储为 **14 维实数向量** (直接在超参数上下界内生成), 仅在交叉/变异时根据参数类型动态转换编码方式, 避免固定维度的编码划分。**1. 种群存储层**: 每个个体为 14 维实数向量, 直接落在各超参数的预设上下界 $[lb_j, ub_j]$ 内, 表示为:

$$\text{Ind}_i = [x_1, x_2, \dots, x_{14}], \quad x_j \in [lb_j, ub_j] \quad (2)$$

其中 lb_j 和 ub_j 为第 j 维超参数的下界和上界。**2. 进化操作层**: 交叉/变异时根据参数类型动态选择编码方式, 设计函数识别参数类型 (0= 连续型, 1= 整数型, 2= 分类型):

- 对**连续型参数** (如学习率、dropout 概率): 保持实数形态, 采用多项式交叉 (SBX) 和多项式变异。
- 对**整数型参数** (如 batch_size、滤波器数量) 和**分类型参数** (如池化类型、优化器类型): 调用 `binary_encode` 函数转换为二进制字符串, 执行二进制交叉/变异后, 再通过 `binary_decode` 函数转回实数, 最终映射为实际参数值。

3. CNN-BiLSTM 参数类型与编码方式:

表 1: CNN-BiLSTM 超参数类型与进化编码方式

维度	超参数名称	参数类型	下界 lb	上界 ub	进化操作编码方式
1	batch_size	整数型	32	256	二进制编码
2	learn_rate	连续型	1e-6	5e-2	保持实数
3	pool_type	分类型	1	2	二进制编码
4	num_filters1	整数型	32	512	二进制编码
5	filter_size1	整数型	1	7	二进制编码
6	lstm_units1	整数型	32	512	二进制编码
7	lstm_units2	整数型	16	256	二进制编码
8	reg_type	分类型	1	3	二进制编码
9	dropout_prob1	连续型	0.1	0.55	保持实数
10	dropout_prob2	连续型	0.1	0.55	保持实数
11	optimizer_type	分类型	1	3	二进制编码
12	num_conv_layers	整数型	1	2	二进制编码
13	conv_dropout	连续型	0.1	0.55	保持实数
14	activation_function	分类型	1	4	二进制编码

4. 超参数映射规则:

- **连续型参数** (维度 2、9、10、13): 种群中存储的实数直接作为参数值, 无需额外映射 (代码通过 `filter_invalid_hyperparam` 确保在 $[lb, ub]$ 内)。
- **整数型参数** (维度 1、4、5、6、7、12): 二进制解码后取整, 其中 batch_size 通过 `filter_invalid_hyperparam` 确保为 2 的幂次。
- **分类型参数** (维度 3、8、11、14): 二进制解码后取整, 映射为具体类别 (如 `pool_type = 1` $\rightarrow$ MaxPool, `2` $\rightarrow$ AvgPool; `optimizer_type = 1` $\rightarrow$ Adam, `2` $\rightarrow$ SGD, `3` $\rightarrow$ RMSprop)。

5. 以下为一个完整的 CNN-BiLSTM 超参数个体编码示例：, 包含种群实数向量、各维度编码过程及最终进化操作所需的编码向量：该超参数组合在种群中被随机生成存储为以下实数向量（在 $[lb, ub]$ 内）：

$$[64.0, 0.001, 1.0, 128.0, 3.0, 256.0, 128.0, 2.0, 0.3, 0.25, 1.0, 2.0, 0.2, 2.0]$$

在交叉/变异阶段，对整数型、分类型参数执行二进制编码，连续型参数保留原值，最终得到进化操作所需的编码向量）：

$$[00100000, 0.001, 0, 001100000, 010, \\ 011100000, 01110000, 01, 0.3, 0.25, \\ 00, 1, 0.2, 01]$$

如第一个编码向量值 00100000，代表 batch size=64。第二个编码向量值 0.001，代表学习率设置为 0.001

2.3 适应度评估

适应度评估流程：

- 1) 解码个体编码为实际超参数配置
- 2) 构建对应模型架构并初始化权重（随机初始化）
- 3) 使用训练集 D_{train} 训练模型**最多 20 轮**（早停策略：验证损失 5 轮不下降则提前终止）
- 4) 在验证集 D_{valid} 上计算 RMSE 和模型参数量 Complexity
- 5) 将适应度值 (Complexity, RMSE) 写入个体

一、交叉操作（Crossover） 交叉操作针对父代种群中随机配对的个体，按参数类型执行差异化策略。交叉率 pc 由 `dynamic_pc` 函数动态调整，基础值为 0.8，根据进化阶段和模型性能微调（前期高交叉率促进探索，后期降低以精细化）。

1. 连续型参数：模拟二进制交叉(SBX) 适用于维度 2(learn_rate)、9(dropout_prob1)、10(dropout_prob2)、13(conv_dropout)。SBX 交叉通过生成分布因子 β 来控制子代与父代的相似度，确保子代在父代附近分布，同时保持多样性。交叉指数 η_c 由 `dynamic_eta_c` 函数动态计算：

$$\eta_c = 5 + 20 \times \text{stage_coef} \times \text{sens_coef}$$

其中 $\text{stage_coef} = 0.3$ （前 30% 代）或 0.7 （后 70% 代）， sens_coef 基于参数敏感性（e.g., learn_rate 的 $\text{sens}=1.0$ ，取 0.4；dropout 的 $\text{sens}=0.7$ ，取 0.7）。 $\eta_c \in [3, 30]$ 。

SBX 交叉公式：对于两个父代参数值 $x_1^{(1)}$ 和 $x_1^{(2)}$ (假设 $x_1^{(1)} \leq x_1^{(2)}$)，生成随机数 $u \sim U(0,1)$ ，计算：

$$\beta = \begin{cases} (2u)^{1/(\eta_c+1)} & \text{if } u \leq 0.5 \\ [1/(2(1-u))]^{1/(\eta_c+1)} & \text{if } u > 0.5 \end{cases}$$

然后子代参数为：

$$x_2^{(1)} = 0.5 \left[(1 + \beta)x_1^{(1)} + (1 - \beta)x_1^{(2)} \right], \quad x_2^{(2)} = 0.5 \left[(1 - \beta)x_1^{(1)} + (1 + \beta)x_1^{(2)} \right]$$

若子代超出 $[lb, ub]$ ，则通过函数截断或重采样以确保边界约束。

2. 整数/分类型参数：二进制单点交叉 适用于维度 1 (batch_size)、3 (pool_type) 等 10 个维度。首先，通过 `binary_encode` 将实数值转换为二进制字符串（位数基于范围，例如 batch_size [32,256] 需 8 位）。随机选择交叉点 k ($1 \leq k < \text{bits}$)，交换 k 之后的子串。交叉后，通过 `binary_decode` 转回实数并映射 (e.g., pool_type: 1→MaxPool, 2→AvgPool)。若解码值无效，使用 `filter_invalid_hyperparam` 重映射到最近合法值 (e.g., batch_size 强制为 2^n)。

二、变异操作 (Mutation) 变异操作针对交叉后的子代个体，按参数类型执行差异化策略。基础变异率 $pm = 0.07$ ，由 `dynamic_pm` 函数调整：前期 pm 高 (0.1) 促进多样性，后期低 (0.05) 避免破坏优质解。

1. 连续型参数：多项式变异 适用于相同 4 个连续型维度。变异指数 η_m 由 `dynamic_eta_m` 函数动态计算：

$$\eta_m = 5 + 7 \times \left(1 - 0.5 \times \frac{g}{G} \right) \times (1.5 - \text{sens}_j)$$

其中 g/G 为进化进度， $\text{sens}_j \in [0, 1]$ (高敏感参数 η_m 小，促进大变异)。 $\eta_m \in [3, 25]$ 。

多项式变异公式：对于参数 x ，生成 $u \sim U(0,1)$ ，计算变异因子 δ ：

$$\delta = \begin{cases} (2u)^{1/(\eta_m+1)} - 1 & \text{if } u < 0.5 \\ 1 - [2(1-u)]^{1/(\eta_m+1)} & \text{if } u \geq 0.5 \end{cases}$$

然后变异后参数：

$$x' = x + \delta \times (ub - lb)$$

若 x' 超出 $[lb, ub]$ ，截断以符合边界要求。

2. 整数/分类型参数：二进制位翻转变异 适用于相同 10 个维度。将参数转换为二进制字符串，按变异概率逐位翻转。变异后解码并映射，若无效则重试或回退。

2.3.1 折中最优解的选择策略

在得到 60 代进化后的最终 Pareto 最优解集后, 需要从该解集中进一步选取一个用于最终模型训练与测试的**折中最优解**。由于不同解在预测误差 (RMSE) 与模型复杂度 (Complexity) 之间的权衡不同, 因此需要构建一个综合指标, 以同时衡量两个目标的相对优劣。

本文采用**归一化欧氏距离最小化**作为折中准则。设最终 Pareto 前沿包含 K 个候选解, 它们对应的 RMSE 为 $f_1^{(k)}$, 复杂度为 $f_2^{(k)}$ 。首先对两个目标分别归一化:

$$\tilde{f}_1^{(k)} = \frac{f_1^{(k)} - \min(f_1)}{\max(f_1) - \min(f_1)}, \quad \tilde{f}_2^{(k)} = \frac{f_2^{(k)} - \min(f_2)}{\max(f_2) - \min(f_2)}$$

再计算每个解的折中评分 (Trade-off Score):

$$\text{Score}^{(k)} = \sqrt{\tilde{f}_1^{(k)2} + \tilde{f}_2^{(k)2}}$$

分数越小的解表示同时具有更小的预测误差与更低的复杂度, 即为折中最优解:

$$k^* = \arg \min_k \text{Score}^{(k)}$$

最终选取的超参数组合即为模型最终训练阶段采用的配置。(在训练最终模型时, 扩大 epoch 为 200)

2.4 并行化优化架构

为充分利用 GPU 资源, 系统采用**多进程并行架构**:

- 每个模型类型 (CNN-BiLSTM、BiLSTM、GRU、CNN、Transformer) 在独立进程中运行

2.5 实验结果与分析

2.5.1 模型性能对比

使用 Sotavento 风电场数据集 (特征包括: 风向、理论功率曲线、有功功率、风速), 按 6:1:1 划分为训练集、验证集和测试集。各模型在测试集上的性能对比如表 2 所示:

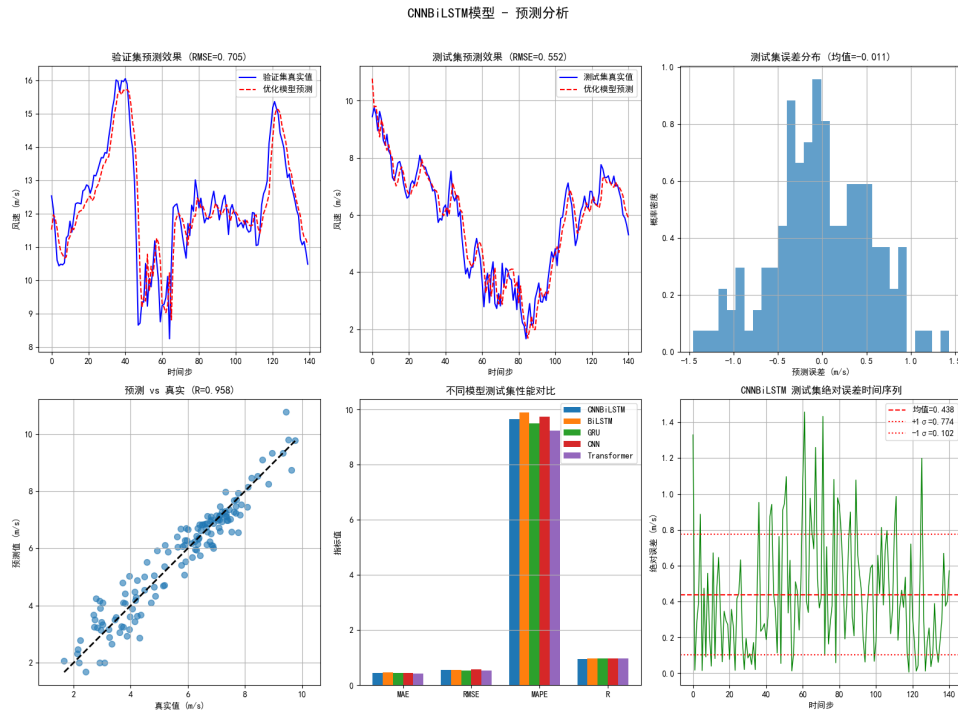


表 2: 各模型测试集性能对比

模型	MAE (m/s)	RMSE (m/s)	MAPE (%)	R
CNN	0.4483	0.5677	9.74	0.9595
GRU	0.4372	0.5281	9.51	0.9639
BiLSTM	0.4559	0.5460	9.89	0.9026
CNN-BiLSTM	0.4380 (3)	0.5518(4)	9.66(3)	0.9580(4)
Transformer	0.4268	0.5303	9.23	0.9630

2.5.2 Pareto 前沿演化分析

图 1展示了 CNN-BiLSTM 模型在 60 代内的 Pareto 前沿演化过程。初始代（第 1 代）的解分布分散，性能较差（ $RMSE > 1.2 \text{ m/s}$ ）；经过 20 代进化，前沿向低 RMSE 区域显著收敛；第 40-60 代前沿趋于稳定，最终得到的 Pareto 最优解集在 RMSE 和复杂度之间实现了良好权衡。

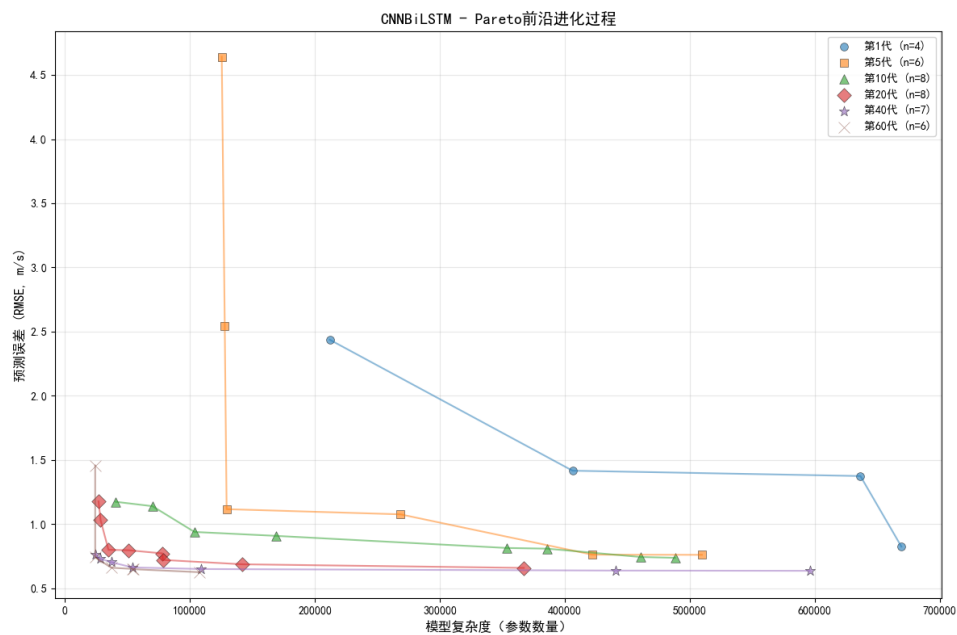


图 1: CNN-BiLSTM 模型 Pareto 前沿演化过程 (第 1, 5, 10, 20, 40, 60 代)

2.5.3 特征权重分析

CNN-BiLSTM 模型的特征加权层学习到的权重分布如图 2 所示。

该结果表明模型能够自适应地识别关键特征，避免人工特征选择的偏差。

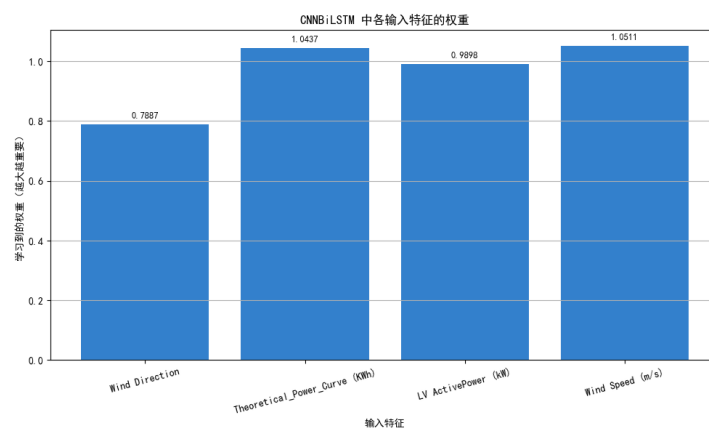


图 2: CNN-BiLSTM 学习到的特征权重分布

2.5.4 模型复杂度与性能权衡

不同模型的复杂度-性能对比如图所示。经过多目标算法优化，很好实现了模型的轻量化。

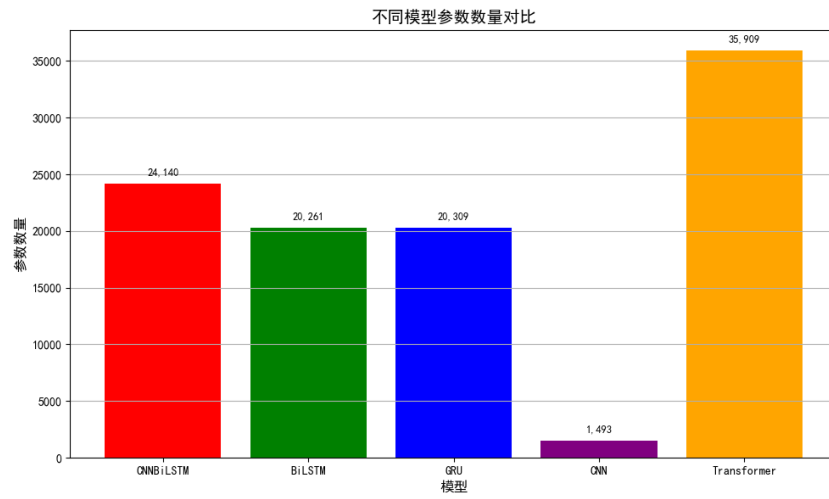


图 3: 模型复杂度与预测性能的 Pareto 前沿